

Rapport de Projet Académique

Cours : Apprentissage Statistique

Détection du cancer du sein

Classification supervisée et comparaison de modèles



Projet réalisé dans le cadre du cours d'Apprentissage Statistique

Réalisé par :

Skander Trigui
Aref Bakali
Youssef Karoui
Siwar Ben Mustapha
Ikram Saidane

Encadré par :

Mme Sana Louhichi
Professeur

Table des matières

Résumé	2
1 Introduction	2
1.1 Contexte	2
1.2 Problématique	2
1.3 Organisation du rapport	2
2 Instructions d'exécution et environnement	2
2.1 Fichiers nécessaires	2
2.2 Logiciels et bibliothèques	2
2.3 Imports principaux	2
3 Présentation du jeu de données	3
3.1 Description générale	3
3.2 Chargement et recodage de la cible	3
4 Analyse exploratoire des données	3
4.1 Statistiques descriptives	3
4.2 Matrice de corrélation	3
4.3 Distribution des variables par classe	4
5 Préparation des données	4
5.1 Séparation train/test	4
5.2 Standardisation et pipelines	5
6 Modélisation détaillée	5
6.1 Principe général	5
6.2 Baseline DummyClassifier	5
6.3 Régression Logistique	5
6.4 Naive Bayes	5
6.5 RidgeClassifier	6
6.6 Lasso	7
6.7 K plus proches voisins	7
6.8 MLP	8
6.9 Étude technique des modèles : coût, optimisation et hyperparamètres	9
6.10 Synthèse de la validation croisée	9
7 Évaluation sur le jeu de test	9
7.1 Métriques utilisées et justification	9
7.2 Tableau comparatif des performances	10
7.3 Matrices de confusion	10
7.4 Courbes ROC et comparaison graphique	10
8 Comparaison approfondie des modèles	10
8.1 Classement final	10
8.2 Interprétation du classement	12
8.3 Comparaison validation croisée et test final	12
9 Analyse du seuil de décision	12
10 Interprétation globale des variables	13
11 Conclusion	13
12 Perspectives	13

Résumé

Ce rapport présente une étude complète de classification binaire appliquée à la détection du cancer du sein à partir de variables biomédicales. Le jeu de données contient 116 observations et 9 variables explicatives : **Age**, **BMI**, **Glucose**, **Insulin**, **HOMA**, **Leptin**, **Adiponectin**, **Resistin** et **MCP.1**. La variable cible **Classification** distingue les patientes saines des patientes atteintes de cancer.

L'objectif est de comparer six modèles d'apprentissage statistique. Dans la version détaillée du notebook, chaque modèle possède sa propre cellule de code, sa propre recherche d'hyperparamètres par **GridSearchCV**, son score de validation croisée et un graphique interprétable. La comparaison finale repose sur des métriques adaptées au contexte médical : sensibilité, spécificité, F1-score, AUC-ROC, faux négatifs et faux positifs. Sur le jeu de test, K-NN est le meilleur compromis avec une accuracy de 0,7917, une sensibilité de 0,7692, une spécificité de 0,8182 et un F1-score de 0,8000. En validation croisée, la Régression Logistique arrive première, ce qui rappelle que les résultats doivent rester prudents à cause de la petite taille du dataset.

1 Introduction

1.1 Contexte

Le cancer du sein est une maladie pour laquelle le dépistage précoce est important. Dans ce projet, l'objectif n'est pas de remplacer un diagnostic médical, mais d'étudier comment des modèles d'apprentissage statistique peuvent distinguer deux groupes à partir de variables biomédicales.

Le problème est formulé comme une classification binaire : la classe 0 correspond aux patientes saines et la classe 1 correspond aux patientes atteintes de cancer.

1.2 Problématique

La problématique principale est la suivante :

Parmi plusieurs modèles de classification supervisée, lequel offre le meilleur compromis entre détection des cas de cancer et limitation des fausses alertes sur un petit jeu de données biomédicales ?

Cette problématique impose de ne pas se limiter à l'accuracy. Dans un contexte médical, les faux négatifs sont particulièrement importants, car ils représentent des cas de cancer non détectés. La sensibilité, la spécificité, le F1-score et les matrices de confusion doivent donc être analysés ensemble.

1.3 Organisation du rapport

Le rapport suit la même logique que le notebook exécuté : instructions d'exécution, environnement, analyse exploratoire, préparation des données, modélisation détaillée modèle par modèle, étude technique des fonctions de coût et des hyperparamètres, validation croisée, évaluation sur le test, comparaison approfondie, analyse du seuil de décision, interprétation globale des variables, conclusion et perspectives.

2 Instructions d'exécution et environnement

2.1 Fichiers nécessaires

Pour exécuter le notebook, il faut disposer des fichiers suivants :

- le notebook Python : `ApprentissageSTAT_cancer_sein_final_detaillée_updated.ipynb` ;
- le fichier de données : `dataR2.csv` ou `dataR2.xlsx`.

Le notebook cherche le fichier dans le dossier courant, puis dans `/content/` pour Google Colab, puis dans `/mnt/data/` pour l'environnement local. Ce chargement portable rend le notebook utilisable dans plusieurs environnements.

2.2 Logiciels et bibliothèques

Les logiciels recommandés sont Python 3.x, Jupyter Notebook ou Google Colab, VS Code/PyCharm si besoin, et LaTeX pour générer le rapport. Les bibliothèques utilisées sont **pandas**, **numpy**, **matplotlib**, **seaborn**, **scikit-learn** et **openpyxl**.

```
# Installation si nécessaire
!pip install pandas numpy matplotlib seaborn scikit-learn openpyxl
```

2.3 Imports principaux

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV, RepeatedStratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression, RidgeClassifier
```

```

from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score, precision_score
from sklearn.metrics import recall_score, f1_score, roc_auc_score, confusion_matrix, matthews_corrcoef

```

3 Présentation du jeu de données

3.1 Description générale

Le jeu de données contient 116 observations et 10 colonnes : 9 variables explicatives et une variable cible. Les variables explicatives sont des mesures biomédicales et physiologiques.

TABLE 1 – Variables utilisées dans le projet

Variable	Type	Description
Age	Numérique	Age de la patiente
BMI	Numérique	Indice de masse corporelle
Glucose	Numérique	Niveau de glucose
Insulin	Numérique	Niveau d'insuline
HOMA	Numérique	Indicateur lié à la résistance à l'insuline
Leptin	Numérique	Niveau de leptine
Adiponectin	Numérique	Niveau d'adiponectine
Resistin	Numérique	Niveau de résistine
MCP.1	Numérique	Protéine MCP-1
Classification	Binaire	0 : saine, 1 : cancer

3.2 Chargement et recodage de la cible

La variable cible originale est recodée en 0 et 1 afin d'être compatible avec les modèles de classification.

```

possible_paths = [Path("dataR2.csv"), Path("dataR2.xlsx"),
                  Path("/content/dataR2.csv"), Path("/content/dataR2.xlsx"),
                  Path("/mnt/data/dataR2.csv"), Path("/mnt/data/dataR2.xlsx")]
data_path = next((p for p in possible_paths if p.exists()), None)
if data_path is None:
    raise FileNotFoundError("Fichier dataR2.csv ou dataR2.xlsx introuvable.")
df = pd.read_csv(data_path) if data_path.suffix.lower() == ".csv" else pd.read_excel(data_path)
df["Classification"] = df["Classification"].replace({1: 0, 2: 1})

```

TABLE 2 – Répartition de la variable cible

Classe	Effectif	Pourcentage
Sain	52	44,83%
Cancer	64	55,17%

La classe Cancer est légèrement majoritaire. Cette répartition n'est pas extrêmement déséquilibrée, mais elle justifie l'utilisation de métriques comme la balanced accuracy, la sensibilité et la spécificité.

4 Analyse exploratoire des données

4.1 Statistiques descriptives

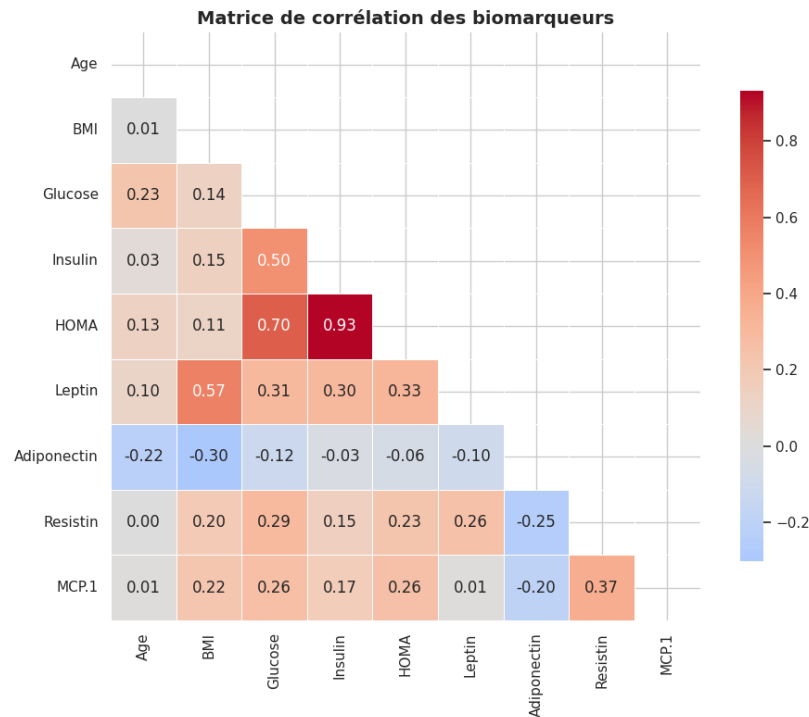
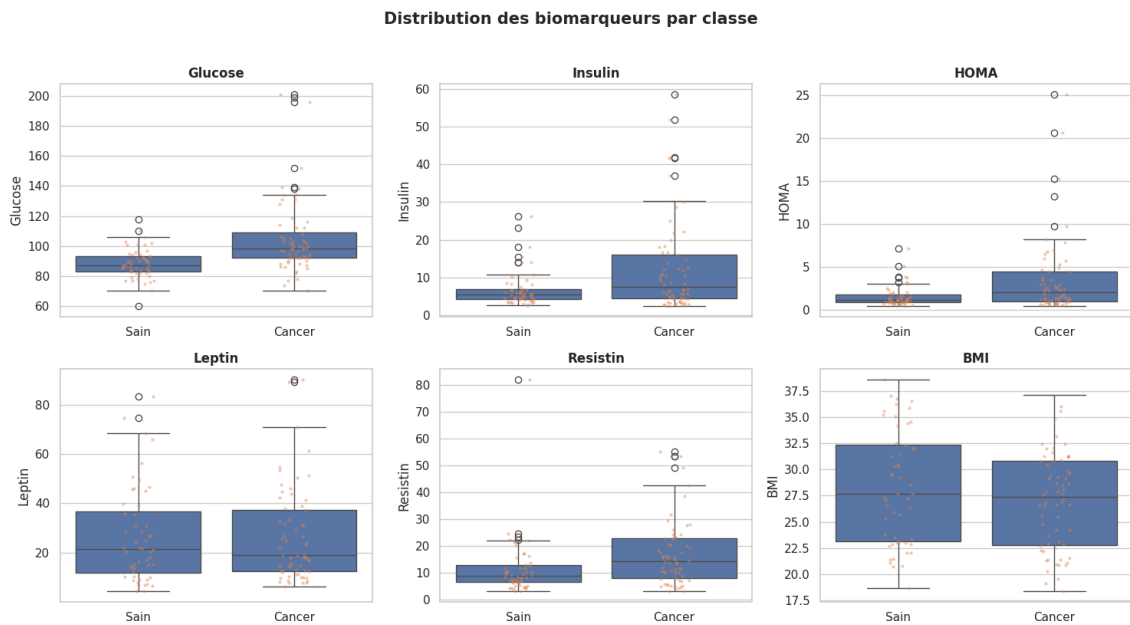
Le tableau suivant résume les principales statistiques descriptives. Aucune valeur manquante n'a été détectée, ce qui évite une étape d'imputation.

TABLE 3 – Statistiques descriptives principales

Variable	Count	Mean	Std	Min	Q1	Median	Q3	Max
Age	116	57,302	16,113	24,000	45,000	56,000	71,000	89,000
BMI	116	27,582	5,020	18,370	22,973	27,662	31,241	38,579
Glucose	116	97,793	22,525	60,000	85,750	92,000	102,000	201,000
Insulin	116	10,012	10,068	2,432	4,359	5,924	11,189	58,460
HOMA	116	2,695	3,642	0,467	0,918	1,381	2,858	25,050
Leptin	116	26,615	19,183	4,311	12,314	20,271	37,378	90,280
Adiponectin	116	10,181	6,843	1,656	5,474	8,353	11,816	38,040
Resistin	116	14,726	12,391	3,210	6,882	10,828	17,755	82,100
MCP.1	116	534,647	345,913	45,843	269,978	471,322	700,085	1698,440

4.2 Matrice de corrélation

La corrélation forte entre *Insulin* et *HOMA* est attendue, car *HOMA* est lié à l'insuline et au glucose. Cette colinéarité peut influencer certains modèles linéaires, ce qui justifie l'utilisation de modèles régularisés comme Ridge et Lasso.

**FIGURE 1** – Matrice de corrélation des variables biomédicales**FIGURE 2** – Distribution de variables biomédicales par classe

4.3 Distribution des variables par classe

Les boxplots montrent que certaines variables, notamment **Glucose**, **Insulin**, **HOMA** et **Resistin**, présentent des différences entre les patientes saines et les patientes atteintes de cancer. Toutefois, les distributions restent largement chevauchées. Aucun biomarqueur isolé ne permet donc une séparation parfaite entre les deux classes.

5 Préparation des données

5.1 Séparation train/test

La base est séparée en 80% pour l'entraînement et 20% pour le test. La séparation est stratifiée pour respecter la proportion des classes.

```
X = df[features]
y = df[target]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
```

Les ensembles obtenus contiennent 92 observations pour l'entraînement et 24 observations pour le test, avec 11 observations saines et 13 observations cancer dans le test. Cette taille réduite est une limite importante : une seule erreur modifie fortement les métriques.

5.2 Standardisation et pipelines

La standardisation est intégrée directement dans les pipelines. Cela évite la fuite de données pendant la validation croisée, car le scaler est ajusté uniquement sur les folds d'entraînement.

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000))
])
```

6 Modélisation détaillée

6.1 Principe général

La partie modélisation du notebook a été réorganisée pour que chaque modèle possède une cellule spécifique. Pour chaque modèle, le notebook contient :

- la définition du pipeline ;
- une grille d'hyperparamètres ;
- une recherche par `GridSearchCV` ;
- le score moyen de validation croisée ;
- un graphique interprétable lié au modèle.

La validation croisée utilisée est une validation stratifiée répétée : 5 folds répétés 5 fois. La métrique d'optimisation est la `balanced_accuracy`, car elle tient compte à la fois de la sensibilité et de la spécificité.

```
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=42)
scoring = "balanced_accuracy"
```

6.2 Baseline DummyClassifier

Une baseline est utilisée pour vérifier que les modèles apprennent réellement mieux qu'une règle naïve. Elle ne fait pas partie des six modèles principaux.

```
baseline = DummyClassifier(strategy="most_frequent", random_state=42)
baseline.fit(X_train, y_train)
y_pred_baseline = baseline.predict(X_test)
```

TABLE 4 – Performance de la baseline DummyClassifier

Modèle	Accuracy	Bal. Acc.	Précision	Recall	Spéc.	F1	FP
Baseline	0,5417	0,5000	0,5417	1,0000	0,0000	0,7027	11

La baseline prédit tous les individus comme Cancer. Elle obtient donc une sensibilité parfaite, mais une spécificité nulle. Cela montre pourquoi le recall seul ne suffit pas pour choisir un bon modèle.

6.3 Régression Logistique

La Régression Logistique est un modèle linéaire adapté à la classification binaire. Elle estime la probabilité qu'une patiente appartienne à la classe Cancer. Dans le notebook, elle est entraînée avec une pénalisation L2 et une recherche sur le paramètre C. Sur le plan technique, elle minimise une fonction de coût de type *log-loss*. Cette minimisation est liée au principe de la descente de gradient, même si `scikit-learn` utilise ici un solveur numérique optimisé comme `lbfgs` plutôt qu'une implémentation manuelle de SGD.

```
logreg_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(penalty="l2", solver="lbfgs",
                                max_iter=2000, class_weight="balanced"))
])
logreg_grid = {"model__C": [0.01, 0.1, 1, 10, 100]}
logreg_search = GridSearchCV(logreg_pipeline, logreg_grid, cv=cv,
                             scoring="balanced_accuracy")
logreg_search.fit(X_train, y_train)
```

Le meilleur score CV obtenu est 0,7604 avec C=1. Le graphique suivant présente les coefficients standardisés de la Régression Logistique.

L'interprétation montre que **Glucose** et **Resistin** ont des coefficients positifs importants, tandis que **Leptin**, **BMI** et **Age** ont des coefficients négatifs dans ce modèle. Ces signes doivent être interprétés comme des associations apprises par le modèle, et non comme des causalités médicales.

6.4 Naive Bayes

Naive Bayes est un modèle probabiliste basé sur le théorème de Bayes. Il suppose que les variables sont indépendantes conditionnellement à la classe. Dans le notebook, `GaussianNB` est utilisé avec une recherche sur `var_smoothing`.

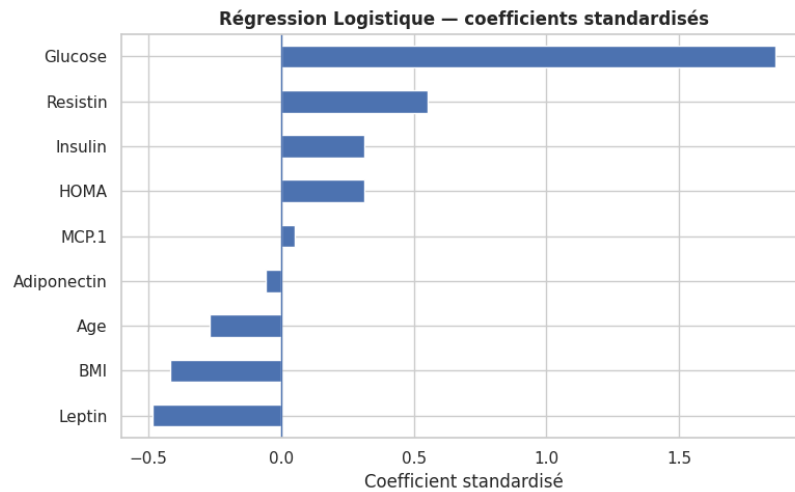


FIGURE 3 — Régression Logistique : coefficients standardisés

```
nb_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", GaussianNB())
])
nb_grid = {"model__var_smoothing": np.logspace(-12, -6, 7)}
nb_search = GridSearchCV(nb_pipeline, nb_grid, cv=cv,
    scoring="balanced_accuracy")
nb_search.fit(X_train, y_train)
```

Le meilleur score CV obtenu est 0,6517 avec `var_smoothing=1e-12`. Le graphique représente la différence des moyennes standardisées entre les patientes Cancer et Sain.

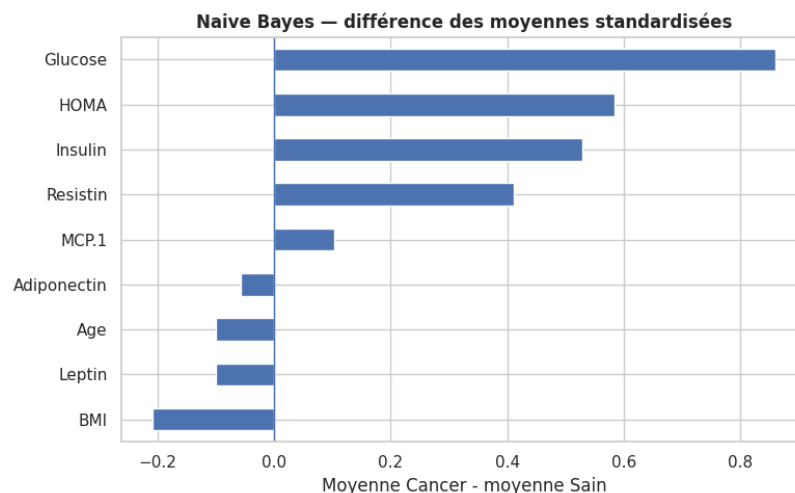


FIGURE 4 — Naive Bayes : différence moyenne standardisée Cancer - Sain

Le graphique indique que **Glucose**, **HOMA**, **Insulin** et **Resistin** sont plus élevés en moyenne dans la classe Cancer. Malgré une bonne AUC sur le test, Naive Bayes présente une sensibilité faible, ce qui limite son intérêt dans un contexte de dépistage.

6.5 RidgeClassifier

RidgeClassifier est un modèle linéaire régularisé par une pénalisation L2. Il est utile lorsque les variables sont corrélées, comme **Insulin** et **HOMA**. Dans le notebook, Ridge est bien représenté par **RidgeClassifier** et non par une deuxième régression logistique L2.

```
ridge_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RidgeClassifier(class_weight="balanced"))
])
ridge_grid = {"model__alpha": np.logspace(-3, 3, 7)}
ridge_search = GridSearchCV(ridge_pipeline, ridge_grid, cv=cv,
    scoring="balanced_accuracy")
ridge_search.fit(X_train, y_train)
```

Le meilleur score CV obtenu est 0,7436 avec `alpha=0,3162`. Le graphique suivant montre les coefficients standardisés appris par Ridge.

Ridge met surtout en avant **Insulin**, **Glucose** et **HOMA**. La régularisation L2 limite les coefficients extrêmes, ce qui rend le modèle plus stable face à la colinéarité.

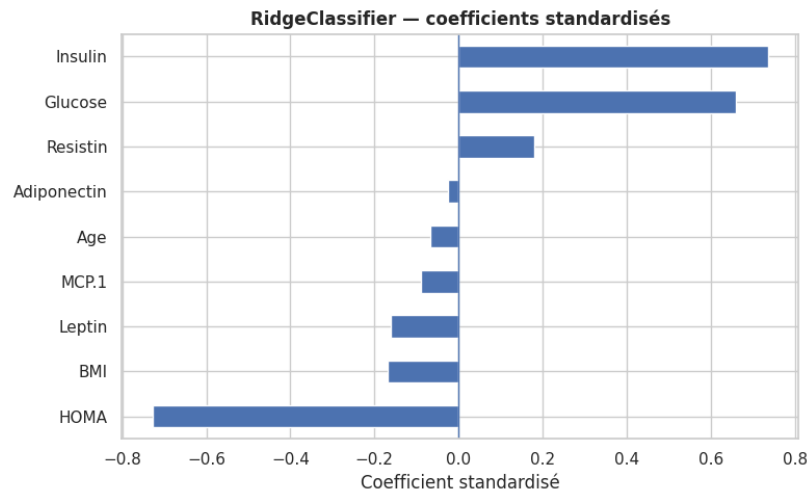


FIGURE 5 — RidgeClassifier : coefficients standardisés

6.6 Lasso

Dans ce projet, Lasso correspond à une Régression Logistique avec pénalisation L1. Cette pénalisation peut réduire certains coefficients exactement à zéro, ce qui permet une sélection implicite de variables.

```
lasso_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(penalty="l1", solver="liblinear",
                                max_iter=3000, class_weight="balanced"))
])
lasso_grid = {"model__C": [0.01, 0.1, 1, 10, 100]}
lasso_search = GridSearchCV(lasso_pipeline, lasso_grid, cv=cv,
                             scoring="balanced_accuracy")
lasso_search.fit(X_train, y_train)
```

Le meilleur score CV obtenu est 0,7546 avec C=1. Lasso retient 6 variables et élimine 3 variables.

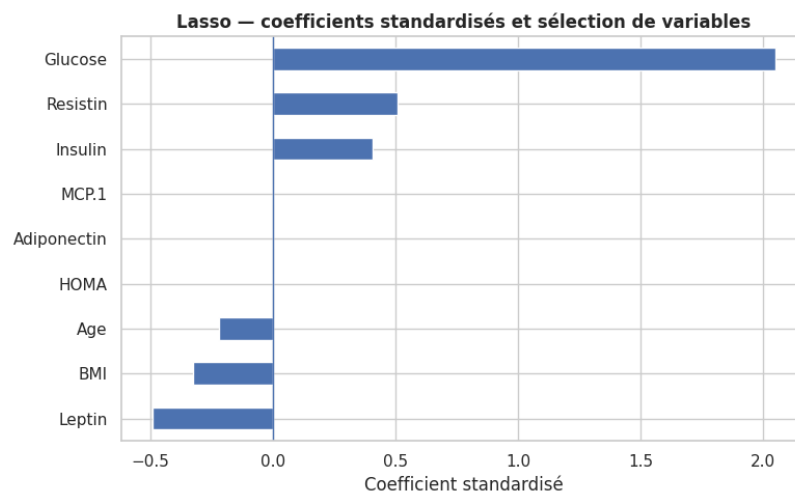


FIGURE 6 — Lasso : coefficients standardisés et sélection de variables

Les variables HOMA, Adiponectin et MCP.1 obtiennent un coefficient nul dans le modèle Lasso. Cette propriété améliore l'interprétabilité, mais elle ne garantit pas une meilleure performance sur le test.

6.7 K plus proches voisins

K-NN est un modèle non paramétrique basé sur la distance entre les observations. Pour classer une nouvelle patiente, il regarde les classes des observations les plus proches. La standardisation est indispensable, car le modèle dépend directement des distances.

```
knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])
knn_grid = {
    "model__n_neighbors": [3, 5, 7, 9, 11, 13, 15],
    "model__weights": ["uniform", "distance"],
    "model__metric": ["euclidean", "manhattan"]
}
knn_search = GridSearchCV(knn_pipeline, knn_grid, cv=cv,
                           scoring="balanced_accuracy")
knn_search.fit(X_train, y_train)
```

Le meilleur score CV obtenu est 0,7130 avec `n_neighbors=11`, `weights=uniform` et `metric=manhattan`. Le graphique montre l'effet du nombre de voisins sur la balanced accuracy.

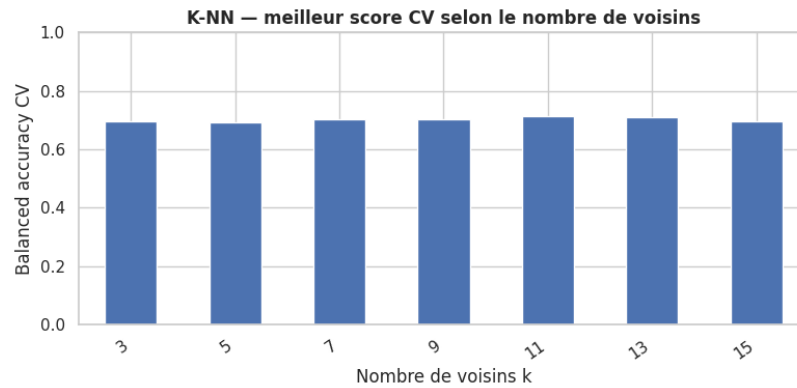


FIGURE 7 — K-NN : influence du nombre de voisins sur la balanced accuracy CV

Le choix `k=11` produit le meilleur score moyen en validation croisée. Même si K-NN n'est pas premier en validation croisée, il devient le meilleur compromis sur le jeu de test final.

Impact de la distance dans K-NN. Le notebook compare également les distances *euclidienne* et *Manhattan*. Cette analyse est importante car K-NN ne construit pas une fonction explicite : sa décision dépend directement de la notion de proximité entre deux observations. La distance euclidienne mesure la distance directe entre deux points, tandis que la distance Manhattan additionne les écarts absolus variable par variable :

$$d_{euclidienne}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad d_{manhattan}(x, y) = \sum_{i=1}^n |x_i - y_i|.$$

Dans ce projet, la distance Manhattan obtient le meilleur score CV pour K-NN, avec une balanced accuracy CV de 0,7130 contre 0,7110 pour la distance euclidienne.

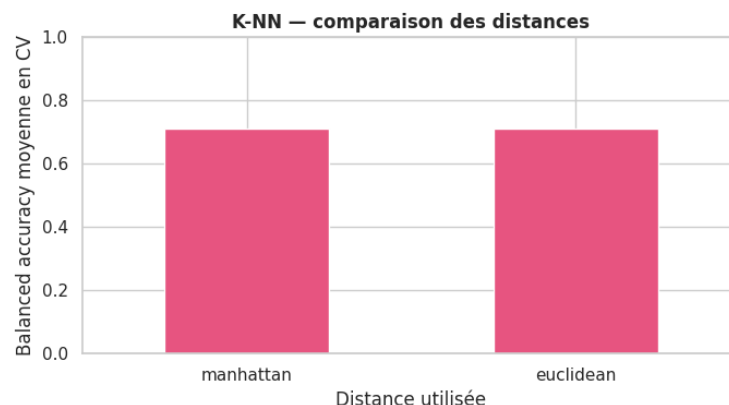


FIGURE 8 — K-NN : comparaison des distances euclidienne et Manhattan

6.8 MLP

Le MLP est un réseau de neurones capable d'apprendre des relations non linéaires entre les variables. Dans le notebook, plusieurs architectures et paramètres d'apprentissage sont testés. Contrairement aux modèles linéaires simples, le MLP apprend ses poids par rétropropagation. La fonction de coût utilisée est une *log-loss* de classification, optimisée par une méthode de type descente de gradient, ici l'algorithme *Adam*.

```
mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", MLPClassifier(max_iter=2000, early_stopping=True,
                           random_state=42))
])
mlp_grid = {
    "model__hidden_layer_sizes": [(16,), (32,), (32, 16)],
    "model__alpha": [0.0001, 0.001, 0.01],
    "model__learning_rate_init": [0.001, 0.01]
}
mlp_search = GridSearchCV(mlp_pipeline, mlp_grid, cv=cv,
                           scoring="balanced_accuracy")
mlp_search.fit(X_train, y_train)
```

Le meilleur score CV obtenu est 0,7367 avec `hidden_layer_sizes=(32,16)`, `alpha=0,001` et `learning_rate_init=0,01`. Le graphique présente la courbe de perte finale.

La loss diminue au fil des itérations, ce qui indique que le modèle apprend. Cependant, avec seulement 116 observations, un réseau de neurones peut manquer de données pour généraliser mieux que les modèles plus simples.

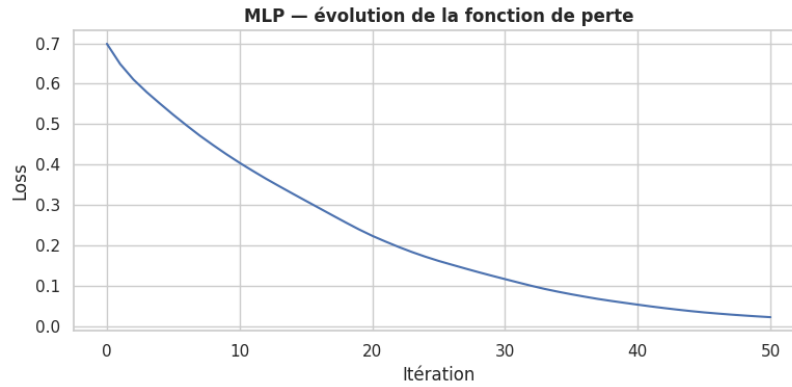


FIGURE 9 – MLP : évolution de la loss durant l'apprentissage

6.9 Étude technique des modèles : coût, optimisation et hyperparamètres

Cette sous-section précise les éléments méthodologiques qui complètent l'étude des modèles : la fonction ou le critère optimisé, la méthode d'optimisation et les hyperparamètres recherchés. Elle est importante car deux modèles peuvent avoir des résultats proches, mais ne reposent pas sur la même logique d'apprentissage.

TABLE 5 – Fonction optimisée, méthode d'apprentissage et hyperparamètres étudiés

Modèle	Fonction ou critère optimisé	Méthode d'optimisation	Hyperparamètres étudiés
Régression Logistique	Minimise une <i>log-loss</i> régularisée L2. Le modèle cherche à estimer la probabilité d'appartenance à la classe Cancer.	Optimisation numérique interne de <code>scikit-learn</code> . Le principe est lié à la descente de gradient, mais l'implémentation utilise un solveur optimisé. Le meilleur modèle est choisi par <code>GridSearchCV</code> .	<code>C</code> , qui contrôle l'intensité de la régularisation. Un petit <code>C</code> impose une régularisation forte ; un grand <code>C</code> laisse plus de liberté au modèle.
Naive Bayes	Estime les probabilités conditionnelles des variables selon chaque classe, avec une hypothèse d'indépendance conditionnelle.	Pas d'optimisation itérative complexe : les moyennes et variances sont estimées directement à partir des données d'apprentissage.	<code>var_smoothing</code> , qui ajoute une petite valeur de lissage à la variance pour améliorer la stabilité numérique.
RidgeClassifier	Optimise un critère linéaire avec pénalisation L2 afin de limiter les coefficients trop élevés.	Optimisation interne du modèle Ridge ; sélection du meilleur niveau de régularisation par validation croisée.	<code>alpha</code> , qui contrôle la force de la pénalisation L2. Plus <code>alpha</code> est grand, plus les coefficients sont réduits.
Lasso	Minimise une <i>log-loss</i> avec pénalisation L1. Cette pénalisation peut mettre certains coefficients exactement à zéro.	Optimisation numérique avec le solveur <code>liblinear</code> , puis sélection par <code>GridSearchCV</code> .	<code>C</code> , qui contrôle l'intensité de la pénalisation L1. Il influence à la fois la performance et le nombre de variables retenues.
K-NN	Ne minimise pas directement une fonction de coût pendant l'entraînement. La prédiction repose sur le vote des voisins les plus proches.	Modèle non paramétrique : l'apprentissage consiste principalement à mémoriser les observations d'entraînement. La prédiction dépend du calcul de distance. Le choix des paramètres est fait par validation croisée.	<code>n_neighbors</code> , <code>weights</code> et <code>metric</code> . Les métriques testées sont notamment <code>euclidean</code> et <code>manhattan</code> .
MLP	Minimise une <i>log-loss</i> de classification par réseau de neurones. L'objectif est de réduire l'écart entre les probabilités prédites et les vraies classes.	Optimisation par descente de gradient avec l'algorithme <code>Adam</code> . L'arrêt anticipé limite le surapprentissage.	<code>hidden_layer_sizes</code> , <code>alpha</code> et <code>learning_rate_init</code> . Ils définissent l'architecture, la régularisation et la vitesse d'apprentissage.

Dans ce projet, le choix des hyperparamètres ne se fait pas manuellement. Pour chaque modèle, une grille de valeurs est testée avec `GridSearchCV`. La validation croisée répétée permet d'estimer la performance moyenne sur plusieurs découpages des données. Le critère de sélection est la `balanced_accuracy`, car elle évite de favoriser un modèle qui prédit trop souvent une seule classe.

```
search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=cv,
    scoring="balanced_accuracy",
    n_jobs=-1
)
search.fit(X_train, y_train)
```

Le recours à `GridSearchCV` permet donc de comparer les modèles de manière plus équitable : chaque modèle est évalué avec ses propres hyperparamètres, mais selon le même protocole de validation croisée et la même métrique principale.

6.10 Synthèse de la validation croisée

En validation croisée, la Régression Logistique est le meilleur modèle. Cependant, le classement sur le jeu de test est différent, ce qui confirme l'instabilité liée à la petite taille du dataset.

7 Évaluation sur le jeu de test

7.1 Métriques utilisées et justification

Les métriques utilisées sont : `accuracy`, `balanced accuracy`, précision, `recall`/sensibilité, spécificité, `F1-score`, `AUC-ROC`, `MCC`, faux négatifs et faux positifs.

TABLE 6 – Classement des modèles en validation croisée répétée

Modèle	Balanced accuracy CV	Meilleurs paramètres
Régression Logistique	0,7604	C=1
Lasso	0,7546	C=1
Ridge	0,7436	alpha=0,3162
MLP	0,7367	alpha=0,001; hidden=(32,16); lr=0,01
K-NN	0,7130	n=11; weights=uniform; metric=manhattan
Naive Bayes	0,6517	var_smoothing=1e-12

Dans un problème médical, l'accuracy seule peut être insuffisante. Un modèle peut obtenir une accuracy correcte tout en ratant des cas de cancer. C'est pourquoi l'évaluation insiste surtout sur :

- le **recall** ou sensibilité, qui mesure la capacité à détecter les patientes atteintes de cancer ;
- le nombre de **faux négatifs**, qui représente les cas de cancer non détectés ;
- la **spécificité**, qui mesure la capacité à reconnaître correctement les patientes saines ;
- le **F1-score**, qui résume l'équilibre entre précision et sensibilité ;
- la **balanced accuracy**, qui combine sensibilité et spécificité ;
- l'**AUC-ROC**, qui mesure la capacité du modèle à séparer les deux classes indépendamment d'un seuil unique.

Ces métriques sont analysées ensemble afin d'éviter de sélectionner un modèle uniquement parce qu'il maximise une seule mesure. En particulier, un recall élevé n'est pas suffisant si la spécificité devient nulle, et une AUC élevée n'est pas suffisante si le seuil choisi produit trop de faux négatifs.

```
def get_positive_scores(model, X_data):
    if hasattr(model, "predict_proba"):
        return model.predict_proba(X_data)[:, 1]
    if hasattr(model, "decision_function"):
        scores = model.decision_function(X_data)
        return (scores - scores.min()) / (scores.max() - scores.min() + 1e-12)
    return model.predict(X_data)
```

7.2 Tableau comparatif des performances

TABLE 7 – Comparaison des six modèles sur le jeu de test

Modèle	Acc.	Bal. Acc.	Précision	Recall	Spéc.	F1	AUC	MCC	FN	FP	Score méd.
K-NN	0,7917	0,7937	0,8333	0,7692	0,8182	0,8000	0,7937	0,5854	3	2	0,7925
Régression Logistique	0,7083	0,7168	0,8000	0,6154	0,8182	0,6957	0,7762	0,4382	5	2	0,7084
MLP	0,7083	0,7168	0,8000	0,6154	0,8182	0,6957	0,7692	0,4382	5	2	0,7077
Lasso	0,7083	0,7168	0,8000	0,6154	0,8182	0,6957	0,7622	0,4382	5	2	0,7070
Naive Bayes	0,6667	0,6853	0,8571	0,4615	0,9091	0,6000	0,8671	0,4063	7	1	0,6641
Ridge	0,6667	0,6783	0,7778	0,5385	0,8182	0,6364	0,7133	0,3671	6	2	0,6594

Le meilleur compromis sur le jeu de test est K-NN. Il obtient la meilleure accuracy, la meilleure balanced accuracy, le meilleur F1-score et le meilleur score médical pondéré. Il commet 3 faux négatifs et 2 faux positifs.

7.3 Matrices de confusion

Les matrices de confusion montrent que K-NN est le modèle le plus équilibré sur le test. Naive Bayes produit peu de faux positifs, mais rate davantage de cancers. En contexte médical, ce point est défavorable.

7.4 Courbes ROC et comparaison graphique

Naive Bayes obtient la meilleure AUC-ROC avec 0,8671. Cependant, son recall cancer est faible : il rate 7 cancers sur 13 dans le test. Cela montre qu'une AUC élevée ne suffit pas à choisir le meilleur modèle si le seuil de classification produit trop de faux négatifs.

Le graphique confirme que K-NN présente le meilleur équilibre global entre recall, spécificité, F1-score et balanced accuracy. Les modèles Régression Logistique, Lasso et MLP ont des performances proches sur le test.

8 Comparaison approfondie des modèles

8.1 Classement final

La comparaison finale ne repose pas sur une seule métrique. Le notebook construit un score médical pondéré pour favoriser les modèles ayant une bonne sensibilité, une bonne spécificité, un bon F1-score et une AUC correcte. Ce score aide à éviter deux erreurs : sélectionner un modèle qui prédit presque tout comme Cancer, ou sélectionner un modèle qui a une bonne AUC mais rate trop de cancers au seuil choisi.

Le score médical utilisé dans le notebook est défini par :

$$\text{Score médical} = 0,40 \times \text{Recall} + 0,30 \times \text{Spécificité} + 0,20 \times \text{F1-score} + 0,10 \times \text{AUC}.$$

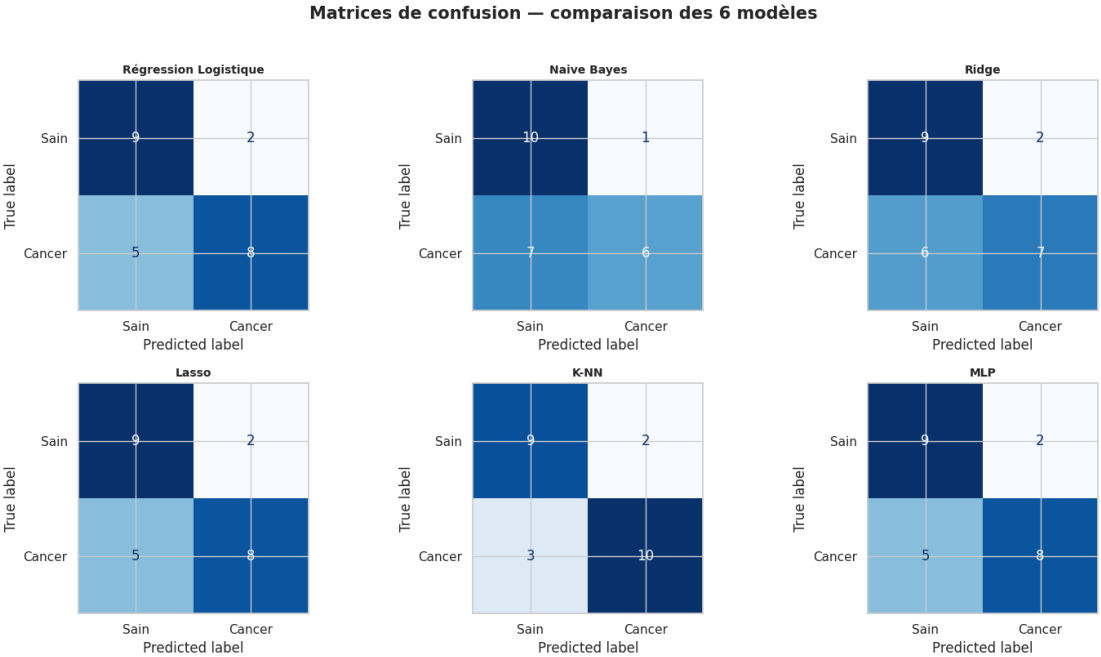


FIGURE 10 – Matrices de confusion des six modèles

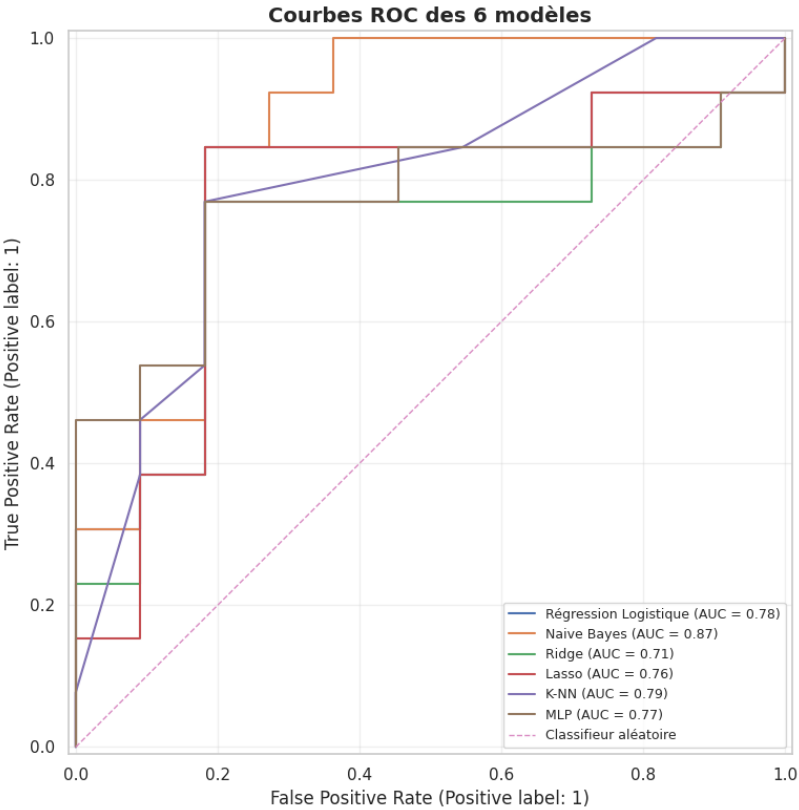


FIGURE 11 – Courbes ROC des six modèles

Le poids le plus élevé est donné au recall, car dans le contexte du cancer du sein, il est prioritaire de réduire les faux négatifs. La spécificité est également fortement pondérée afin de limiter les fausses alertes chez les patientes saines. Le F1-score et l’AUC complètent l’évaluation en apportant une vision d’équilibre et de séparation globale des classes.

TABLE 8 – Classement final selon le score médical pondéré

Rang	Modèle	Score médical	Recall	Spéc.	Interprétation
1	K-NN	0,7925	0,7692	0,8182	Meilleur compromis global sur le test
2	Régression Logistique	0,7084	0,6154	0,8182	Modèle stable et interprétable, mais plus de faux négatifs
3	MLP	0,7077	0,6154	0,8182	Correct, mais pas supérieur aux modèles simples
4	Lasso	0,7070	0,6154	0,8182	Interprétable grâce à la sélection de variables
5	Naive Bayes	0,6641	0,4615	0,9091	Bonne AUC, mais trop de cancers ratés
6	Ridge	0,6594	0,5385	0,8182	Régularisé, mais performance plus faible sur le test

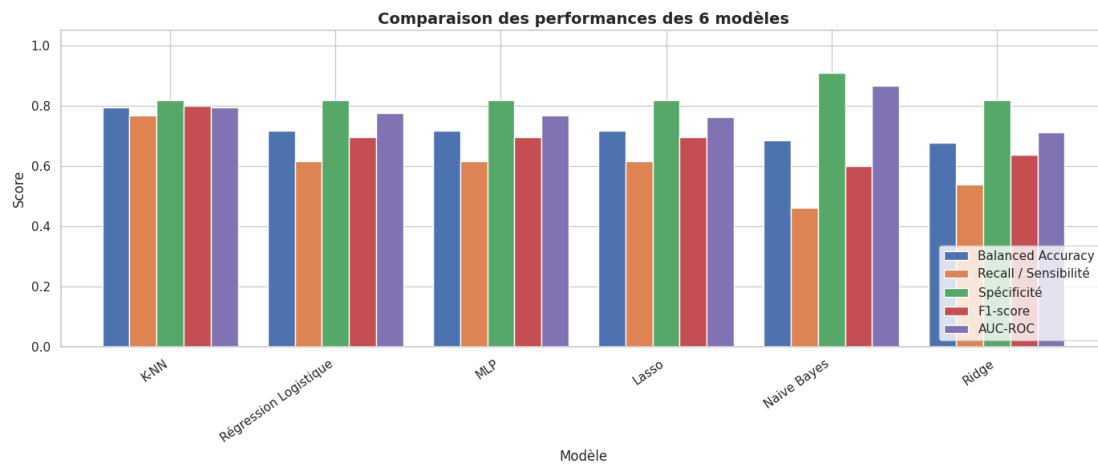


FIGURE 12 – Comparaison graphique des métriques principales

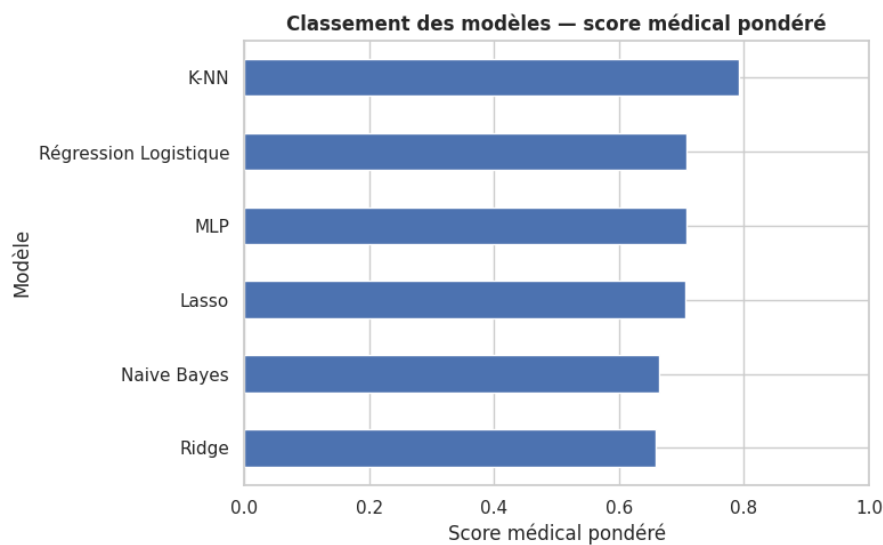


FIGURE 13 – Classement des modèles selon le score médical pondéré

8.2 Interprétation du classement

K-NN est retenu comme meilleur modèle sur le jeu de test, car il combine la meilleure accuracy, le meilleur F1-score et le plus faible nombre de faux négatifs parmi les modèles comparés. Dans ce projet, le nombre de faux négatifs est central : un faux négatif correspond à une patiente classée comme saine alors qu'elle appartient à la classe Cancer.

La Régression Logistique arrive deuxième dans le classement final. Elle reste importante parce qu'elle est première en validation croisée et qu'elle est plus interprétable que K-NN. Cela signifie qu'elle est plus stable sur les folds d'apprentissage, même si elle n'est pas la meilleure sur le test final.

Naive Bayes illustre une situation importante : il obtient la meilleure AUC-ROC, mais il n'est pas choisi comme meilleur modèle. La raison est que son seuil de décision conduit à un recall faible et à 7 faux négatifs. Dans un contexte de dépistage, cette faiblesse est plus grave qu'une simple baisse d'accuracy.

Le MLP ne dépasse pas les modèles plus simples. Ce résultat est cohérent avec la taille réduite du dataset : les réseaux de neurones sont flexibles, mais ils nécessitent souvent plus de données pour généraliser correctement.

8.3 Comparaison validation croisée et test final

Les résultats montrent deux lectures complémentaires :

- en validation croisée, la Régression Logistique obtient la meilleure balanced accuracy moyenne ;
- sur le jeu de test final, K-NN obtient le meilleur compromis global.

Cette différence n'est pas une contradiction. Elle indique que le jeu de données est petit et que les performances sont sensibles au découpage train/test. La conclusion doit donc rester prudente : K-NN est le meilleur modèle sur ce test, mais il faudrait confirmer ce choix sur un échantillon plus large.

9 Analyse du seuil de décision

Par défaut, un modèle classe une observation comme Cancer si la probabilité prédite est supérieure ou égale à 0,5. En médecine, il peut être pertinent d'ajuster ce seuil. Un seuil plus bas augmente la sensibilité, mais augmente aussi les faux positifs.

Le seuil 0,50 donne le meilleur compromis global, avec le meilleur F1-score et la meilleure balanced accuracy. Un seuil

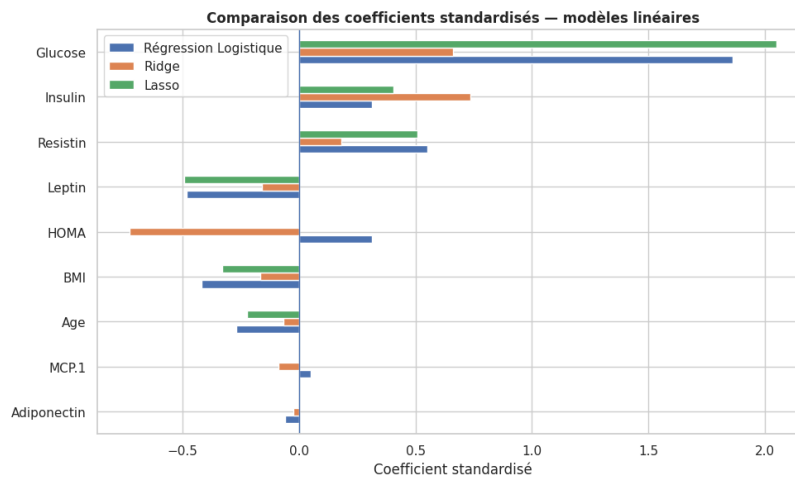
TABLE 9 – Analyse de seuil pour K-NN

Seuil	Recall	Spécificité	F1	Bal. Acc.	FN	FP
0,20	1,000	0,000	0,703	0,500	0	11
0,30	1,000	0,182	0,743	0,591	0	9
0,40	0,846	0,455	0,733	0,650	2	6
0,50	0,769	0,818	0,800	0,794	3	2
0,65	0,462	0,909	0,600	0,685	7	1
0,80	0,385	0,909	0,526	0,647	8	1

plus bas comme 0,40 réduit les faux négatifs, mais augmente fortement les faux positifs. Un seuil plus haut réduit les fausses alertes, mais rate trop de cancers.

10 Interprétation globale des variables

Les modèles linéaires permettent d'analyser les coefficients standardisés. Cette interprétation reste prudente : les coefficients décrivent une relation statistique dans le modèle, pas une causalité médicale.

**FIGURE 14** – Comparaison des coefficients des modèles linéaires

Les variables qui ressortent souvent sont **Glucose**, **Insulin**, **HOMA** et **Resistin**. Lasso élimine trois variables en leur donnant un coefficient nul, ce qui rend le modèle plus parcimonieux. Ridge conserve toutes les variables mais réduit l'amplitude des coefficients grâce à la pénalisation L2.

11 Conclusion

Ce projet a permis de construire une chaîne complète de classification binaire pour la détection du cancer du sein : chargement des données, analyse exploratoire, préparation, modélisation détaillée, étude des fonctions de coût et des hyperparamètres, validation croisée, évaluation sur test, comparaison et analyse du seuil de décision.

Le modèle K-NN est retenu comme meilleur compromis sur le jeu de test. Il obtient les meilleures valeurs d'accuracy, de balanced accuracy et de F1-score, avec 3 faux négatifs et 2 faux positifs. Il est donc plus équilibré que les autres modèles au seuil 0,50.

Cependant, la Régression Logistique est première en validation croisée. Cette différence montre que les résultats sont sensibles au découpage des données. La conclusion doit donc être prudente : K-NN est le meilleur modèle dans cette expérimentation, mais il faudrait confirmer ce choix avec davantage de données et une validation externe.

12 Perspectives

Pour améliorer le projet, plusieurs pistes sont possibles :

- utiliser un jeu de données plus grand afin de réduire l'instabilité des métriques ;
- réaliser une validation externe sur des données indépendantes ;
- tester d'autres méthodes d'ajustement du seuil de décision ;
- analyser plus finement les faux négatifs pour comprendre les profils mal classés ;
- ajouter des méthodes d'interprétabilité comme SHAP ou permutation importance ;
- comparer les résultats avec des modèles d'ensemble, tout en évitant le surapprentissage.